15.8
API Versioning Strategies

Evolving APIs without breaking consumers

Java Backend Architecture Interview Series • Tutorial Edition • Easy Diagrams & Examples

Why API Versioning?

Once you publish an API, clients depend on it. If you change the response format, you break those clients. API versioning lets you evolve your API while keeping old consumers working. This chapter covers the three main strategies and how to retire old versions safely.

■ **Simple Analogy:** Think of a power socket. When countries upgraded from round to flat pins, they didn't rip out all old sockets overnight. They supported both for years, with adapters (translation layers) for a transition period.

Strategy 1: URI Versioning

Put the version in the URL path. Most common, most visible, easiest to route.

```
GET /api/v1/users/123 --> old response format
GET /api/v2/users/123 --> new response format

// Spring controller
@RestController
@RequestMapping("/api/v2/users")
public class UserV2Controller { ... }
```

Best for: public APIs. Used by Stripe, Twilio, GitHub. Easy to cache, route in API gateway.

Strategy 2: Header Versioning

Version is in an HTTP request header. Clean URLs, but harder to test in a browser.

```
GET /api/users/123
Accept: application/vnd.myapi+json;version=2

// Or custom header:
GET /api/users/123
API-Version: 2
```

Best for: internal APIs. Used by Microsoft Graph API. Not cacheable by default.

Strategy 3: Query Parameter Versioning

```
GET /api/users/123?version=2
GET /api/users/123?api-version=2024-01-01 (date-based)
```

Least common. Easy to add but pollutes URLs. OK for simple internal use.

Deprecation Lifecycle

Phase	Action	Duration
Announce	Add Deprecation header to v1 responses	Day 0
Sunset	Add Sunset header with shutdown date	Day 0
Grace	v1 still works; consumers get warnings in logs	6-12 months
Shutdown	v1 returns 410 Gone	After sunset date

```
# Response headers for deprecated v1 endpoint
Deprecation: Sat, 01 Jan 2025 00:00:00 GMT
Sunset: Sat, 01 Jul 2025 00:00:00 GMT
Link: ; rel="successor-version"
```

■ **Real-World Example:** Stripe: Uses date-based versioning ('Stripe-Version: 2024-04-10' header). Each API key is pinned to the version at signup — customers never see breaking changes automatically. This has kept their API backwards-compatible for 10+ years.